

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)



Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 3
по дисциплине: Вычислительная математика
по теме: Решение систем нелинейных уравнений

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: ст. преподаватель:
Четвертухин В. Р.

Белгород 2024

Цель работы: Изучить методы решения систем нелинейных уравнений и особенности их алгоритмизации в экосистемах языков Python и Rust.

Цель работы обуславливает постановку и решение следующих задач:

- 1) Рассмотреть теоретические основы решения систем нелинейных уравнений.
- 2) Научиться выбирать методы и алгоритмизировать решение систем нелинейных уравнений в зависимости от численной ситуации с вниманием к проблемам разрешимости, точности, численной стабильности и эффективности.
- 3) Выполнить индивидуальное задание, закрепляющее на практике полученные знания и практические навыки (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления). Первая часть данного задания предполагает построение графиков нелинейных функций из системы уравнений индивидуального задания и выбор точки начального приближения для нахождения одного корня, ближайшего к началу координат.

Вторая часть задания предполагает изучение алгоритмических техник программы на языке Rust для решения системы нелинейных уравнений демонстрационного и своего индивидуального задания по методу Ньютона в интерактивном блокноте Jupyter.

Третья часть задания предполагает самостоятельное написание двух программ на языке

Rust: для решения той же системы нелинейных уравнений (своего индивидуального задания) по методу простой итерации и по методу градиентного спуска.

Необходимо сравнить вычислительные схемы и полученные результаты для разных алгоритмов между собой.

- 7) Отобразить в отчете все полученные результаты, включая графики, тексты программ.

Сделать выводы.

$$7. \quad \begin{cases} \tanh(x_1) - \sin(x_2) = 0.5 \\ x_1^2 + e^{-x_2} = 3 \end{cases}$$

Первая часть:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import fsolve
import math

def solve_nonlinear_system1(p):
    x1, x2 = p
    return (np.tanh(x1) + np.sin(x2) - 0.5, x1**2 + np.exp(-x2) - 3)

def nonlinear_equations1(variables):
    x1, x2 = variables
    equation1 = np.tan(x1) + np.sin(x2) - 0.5
    equation2 = x1**2 + np.exp(-x2) - 3.
    return [equation1, equation2]

def plot_solution_and_equations1(solution):
    x1_values = np.linspace(-5, 5, 400)
    x2_values = np.linspace(-5, 5, 400)

    X1, X2 = np.meshgrid(x1_values, x2_values)
    Z1 = np.tanh(X1) + np.sin(X2) - 0.5
    Z2 = X1**2 + np.exp(-X2) - 3

    plt.figure(figsize=(8, 6))

    contour1 = plt.contour(X1, X2, Z1, levels=[0], colors='r')
    contour2 = plt.contour(X1, X2, Z2, levels=[0], colors='b')

    plt.clabel(contour1, inline=1, fontsize=10, fmt='Уравнение 1')
    plt.clabel(contour2, inline=1, fontsize=10, fmt='Уравнение 2')

    # Точка решения
    plt.plot(solution[0], solution[1], 'ko') # 'ko' означает черный цвет
    # ('k') и форму точки ('o')
    plt.text(solution[0], solution[1], 'Решение',
    verticalalignment='bottom')

    plt.xlabel('x1')
    plt.ylabel('x2')
    plt.title('Решение системы нелинейных уравнений')
    plt.grid(True)
    plt.show()

# Начальное приближение для поиска решения
initial_guess = [1, 1]

# Решение системы нелинейных уравнений
solution = fsolve(solve_nonlinear_system1, initial_guess)
```

```

# Проверка решения
solution_check = nonlinear_equations1(solution)

print(f"Получено решение: x1 = {solution[0]}, x2 = {solution[1]}")
print(f"Проверка = 0: {solution_check[0]}, {solution_check[1]}")

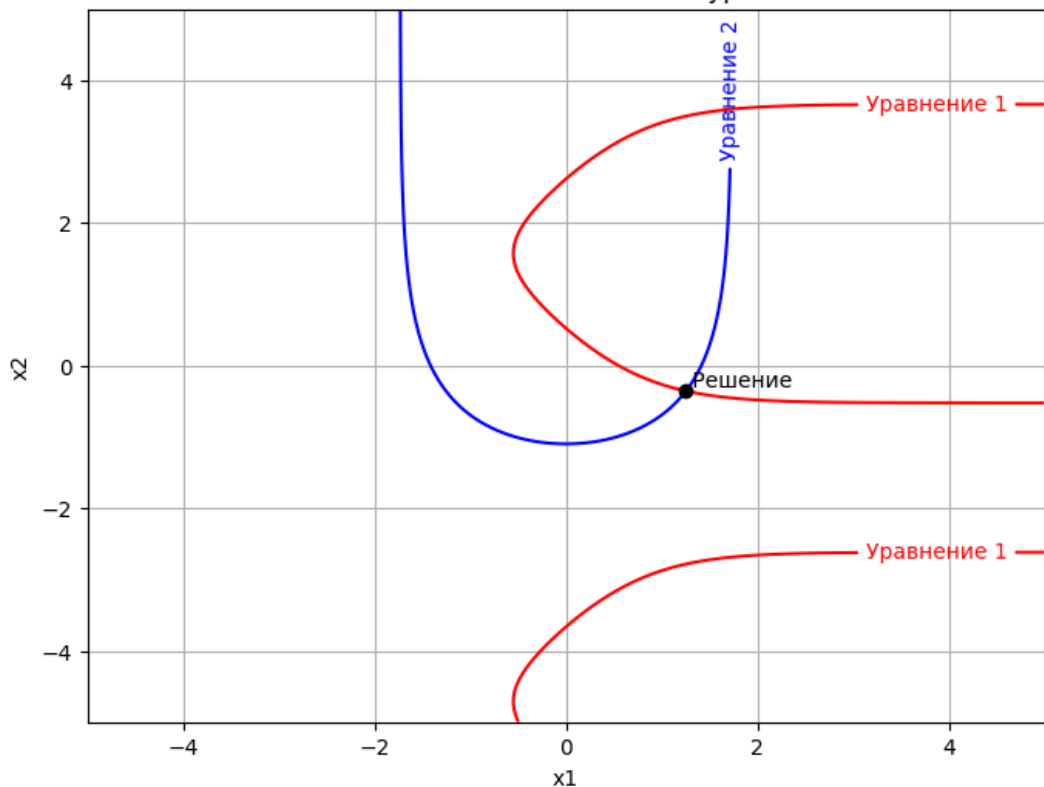
# Построение графика системы уравнений
plot solution and equations1(solution)

```



Получено решение: $x_1 = 1.253503790199433$, $x_2 = -0.3567847114882938$
 Проверка = 0: 2.1959215417829574 , $7.543299318513164e-12$

Решение системы нелинейных уравнений



Вторая часть:

```
%%writefile lab3_ME_newton.rs
use std::f64;

fn is_zero(n: f64, eps: f64) -> bool {
    n.abs() < eps
}

fn inverse_matrix_2x2(matrix: [[f64; 2]; 2], epsilon: f64) ->
Result<[[f64; 2]; 2], &'static str> {
    let det = matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0];
    if is_zero(det, epsilon) {
        return Err("Матрица является вырожденной и не имеет обратной.");
    }
    let inv_det = 1.0 / det;
    let inv_matrix = [
        [matrix[1][1] * inv_det, -matrix[0][1] * inv_det],
        [-matrix[1][0] * inv_det, matrix[0][0] * inv_det],
    ];
    Ok(inv_matrix)
}

fn matrix_vector_multiply(matrix: [[f64; 2]; 2], vector: [f64; 2]) ->
[f64; 2] {
    [
        matrix[0][0] * vector[0] + matrix[0][1] * vector[1],
        matrix[1][0] * vector[0] + matrix[1][1] * vector[1],
    ]
}

fn f(x: [f64; 2]) -> [f64; 2] {
    [
        f64::tanh(x[0]) + f64::sin(x[1]) - 0.5,
        x[0].powi(2) + f64::exp(-x[1]) - 3.0,
    ]
}

fn jacobian(x: [f64; 2]) -> [[f64; 2]; 2] {
    [
        [f64::tanh(x[0]).powi(2), f64::cos(x[1])],
        [2.0 * x[0], -f64::exp(-x[1])],
    ]
}

fn norm(vector: [f64; 2]) -> f64 {
    vector.iter().map(|&v| v.abs()).fold(0.0, f64::max)
}
```

```

fn newton_method(f: fn([f64; 2]) -> [f64; 2], jacobian: fn([f64; 2]) ->
[[f64; 2]; 2], initial_guess: [f64; 2], epsilon: f64, max_iterations:
usize,) -> Result<[f64; 2], &'static str> {
    let mut x = initial_guess;
    for _ in 0..max_iterations {
        let j = jacobian(x);
        let inv_j = inverse_matrix_2x2(j, epsilon)?;
        let fx = f(x);
        let delta = matrix_vector_multiply(inv_j, [-fx[0], -fx[1]]);
        x = [x[0] + delta[0], x[1] + delta[1]];
        if norm(delta) < epsilon {
            return Ok(x);
        }
    }

    Err("Алгоритм не сошелся")
}

fn main() {
    let epsilon = 1e-6; // Задаем значение epsilon
    let initial_guess = [1.0, 1.0];

    match newton_method(f, jacobian, initial_guess, epsilon, 100) {
        Ok(solution) => println!("Решение: {:?}", solution),
        Err(e) => println!("{}", e),
    }
}

```

Решение: [1.2535038178031157, -0.3567846630465124]

Третья часть:

```
use std::f64;

// Функции системы уравнений
fn equation1(x1: f64, x2: f64) -> f64 {
    x1.tanh() + x2.sin() - 0.5
}

fn equation2(x1: f64, x2: f64) -> f64 {
    x1.powi(2) + (-x2).exp() - 3.0
}

// Метод простой итерации
fn simple_iteration_method(mut x1: f64, mut x2: f64, max_iterations:
usize, epsilon: f64) -> (f64, f64) {
    for _ in 0..max_iterations {
        let new_x1 = equation1(x1, x2);
        let new_x2 = equation2(x1, x2);

        if (new_x1 - x1).abs() < epsilon && (new_x2 - x2).abs() < epsilon
{
            break;
        }

        x1 = new_x1;
        x2 = new_x2;
    }
    (x1, x2)
}

// Метод градиентного спуска
fn gradient_descent_method(mut x1: f64, mut x2: f64, max_iterations:
usize, epsilon: f64, learning_rate: f64) -> (f64, f64) {
    for _ in 0..max_iterations {
        let grad_x1 = (equation1(x1 + epsilon, x2) - equation1(x1, x2)) /
epsilon;
        let grad_x2 = (equation2(x1, x2 + epsilon) - equation2(x1, x2)) /
epsilon;

        x1 -= learning_rate * grad_x1;
        x2 -= learning_rate * grad_x2;

        if grad_x1.abs() < epsilon && grad_x2.abs() < epsilon {
            break;
        }
    }
    (x1, x2)
}
```

```

fn main() {
    let epsilon = 0.0001;
    let max_iterations = 1000;
    let learning_rate = 0.01;

    let initial_x1 = 0.0;
    let initial_x2 = 0.0;

    // Решение с помощью метода простой итерации
    let (x1_simple, x2_simple) = simple_iteration_method(initial_x1,
initial_x2, max_iterations, epsilon);
    println!("Решение простым итерационным методом:");
    println!("X1 = {}", x1_simple);
    println!("X2 = {}", x2_simple);

    // Решение с помощью метода градиентного спуска
    let (x1_gradient, x2_gradient) = gradient_descent_method(initial_x1,
initial_x2, max_iterations, epsilon, learning_rate);
    println!("Решение методом градиентного спуска:");
    println!("X1 = {}", x1_gradient);
    println!("X2 = {}", x2_gradient);
}

```

```

Решение простым итерационным методом:
X1 = -0.4288296803058116
X2 = -0.12005045606935916
Решение методом градиентного спуска:
X1 = -2.1355156613294457
X2 = 2.398941029077717

```

Ссылка на блокнот:

<https://colab.research.google.com/drive/1SmgsVR8WdPs9gqpCX8ERpPxjCzB1IMbz?usp=sharing>

Вывод:

Изучение методов решения систем нелинейных уравнений в Python и Rust демонстрирует разные подходы к алгоритмизации, преимущества и недостатки каждого языка. Python предлагает удобство использования и богатый выбор готовых решений, в то время как Rust обеспечивает высокую производительность и безопасность.